Procesamiento Distribuido
Parte 1

Elaborada por: M. en C. Ukranio Coronilla

Una función hash es una función matemática que recibe datos, los “mezcla” y produce un valor compacto llamado hash. Por ejemplo, si tenemos como dato el texto “hola” podríamos crear una función hash muy sencilla que suma los valores ASCII de cada una de sus letras obteniéndose 420. El numero 420 es el hash de “hola” y a partir de ese valor sería muy difícil obtener el dato original.

El SHA-256 es una función hash criptográfica que transforma un texto en una cadena de 256 bits, equivalente a 64 caracteres hexadecimales ($\mathbf{1111_2 = h_{16}}$) y se usa en Bitcoin para generar hashes de bloques. Por ejemplo, para la cadena “hola” su hash SHA-256 es:

`b221d9dbb083a7f33428d7c2a3c3198ae925614d70210e28716ccaa7cd4ddb79`

El siguiente programa usa la clase MessageDigest para encontrar el hash SHA-256 de la palabra “hola”:

TestSHA.java

```java
import java.security.MessageDigest;
import java.nio.charset.StandardCharsets;

public class TestSHA {

    public static void main(String[] args) throws Exception {

        String texto = "hola";

        MessageDigest md = MessageDigest.getInstance("SHA-256");

        byte[] hash = md.digest(texto.getBytes());

        StringBuilder hex = new StringBuilder();

        for(byte b : hash){
            hex.append(String.format("%02x", b));
        }
```

```
        System.out.println(hex.toString());
    }
}
```

Pruebe que no obtenemos el mismo hash para las cadenas “hola”, “Hola” y “HOLA”.

El siguiente programa muestra como alguien que conoce el hash de una contraseña podría intentar descubrir el texto original del cual proviene, mediante un ataque de fuerza bruta para una contraseña sencilla de tres letras minúsculas:

FuerzaBrutaHash.java

```
import java.security.MessageDigest;

public class FuerzaBrutaHash {

    public static void main(String[] args) throws Exception {

        String passwordSecreta = "ipn";

        String hashObjetivo = sha256(passwordSecreta);

        System.out.println("Hash objetivo: " + hashObjetivo);
        System.out.println("Buscando contraseña...\n");

        long inicio = System.currentTimeMillis();

        int intentos = 0;

        for(char a='a'; a<='z'; a++){
            for(char b='a'; b<='z'; b++){
                for(char c='a'; c<='z'; c++){

                    String intento = "" + a + b + c;

                    intentos++;

                    String hash = sha256(intento);

                    if(hash.equals(hashObjetivo)){

                        long fin = System.currentTimeMillis();

                        System.out.println("Contraseña encontrada: " + intento);
                        System.out.println("Intentos: " + intentos);
                        System.out.println("Tiempo(ms): " + (fin - inicio));

                        return;
                    }
                }
            }
        }

        System.out.println("No encontrada");
    }

    public static String sha256(String texto) throws Exception {

        MessageDigest md = MessageDigest.getInstance("SHA-256");

        byte[] hash = md.digest(texto.getBytes());

        StringBuilder hex = new StringBuilder();

        for(byte b : hash){
            hex.append(String.format("%02x", b));
```

```
        }

        return hex.toString();
    }
}
```

Ejercicio 1

Modifique el programa anterior para encontrar en lugar de una contraseña de tres letras minúsculas, una de cinco letras minúsculas. Inicialice con una palabra conocida de cinco letras que inicie con la letra “z” para maximizar el cómputo y tome la captura de pantalla como comprobante de este ejercicio y para tener constancia del tiempo que tarda en encontrar la contraseña.

Ejercicio 2

Siga las instrucciones de la IA para instalar la App Termux y el JDK en su celular Android. Posteriormente envíe, compile y ejecute el código del ejercicio 1 con la misma palabra de búsqueda en su celular. A partir del resultado obtenido determine el **speedup** o factor de aceleración (consultar con la IA el concepto) entre su LAP y el teléfono celular. Adjunte captura de pantalla de la ejecución en su celular como comprobante.

Este tipo de tareas se pueden paralelizar haciendo que un proceso **Master** divida el trabajo en pequeñas tareas, y las envíe a otros procesos **Workers** que procesen las tareas pequeñas. Al final el proceso **Master** obtiene todas las respuestas de los **Workers** y procesa un resultado final. Estos son los principios de las técnicas **MapReduce** que procesan volúmenes masivos de datos.

El **procesamiento distribuido** es un modelo de computación donde varias computadoras (o servidores) trabajan juntas para resolver un mismo problema, dividiendo el trabajo entre ellas y coordinándose a través de una red. Para llevar a cabo el procesamiento distribuido primero programaremos un servidor de procesamiento como se sugiere en el siguiente ejercicio.

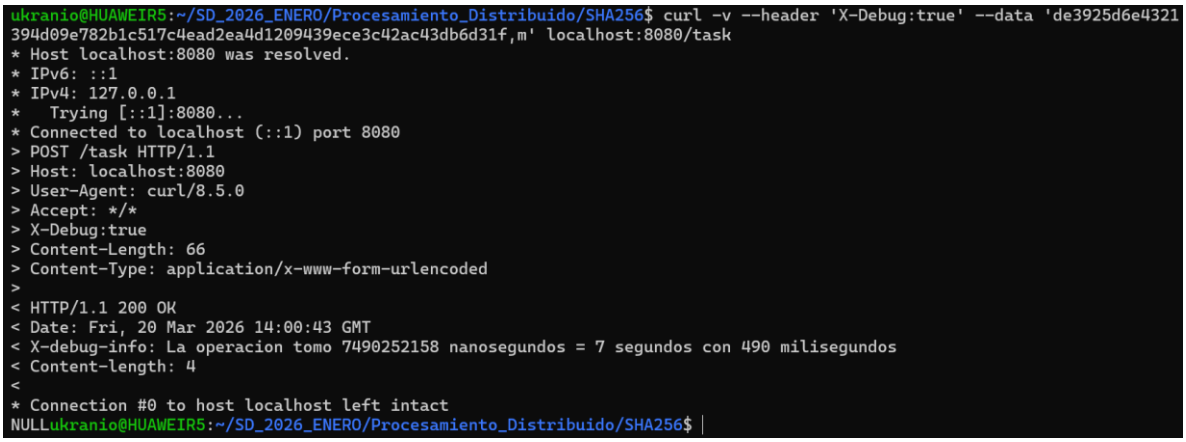
Ejercicio 3

Modifique el servidor web HTTP y adjunte el código del ejercicio 1 para que el endpoint **/task** reciba el `hashObjetivo` junto con la primera de las cinco letras que debe procesar separados por una coma. El servidor devolverá la contraseña en caso de encontrarla o la palabra “NULL” en caso de no encontrarla. Envíe el header

personalizado X-debug-info para obtener el tiempo en segundos y milisegundos que tomó la operación.

En este caso si el servidor recibe la letra n entonces buscará la contraseña en todas las posibles combinaciones desde naaaa hasta nzzzz. Envíe el código y dos capturas de pantalla donde se prueba con CURL cuando se encuentra la palabra y cuando no se encuentra, por ejemplo:

Caso donde no se encuentra la contraseña y devuelve NULL (note que aparece la palabra hasta la línea del prompt):



Caso donde encuentra la contraseña y la devuelve:

